

Solution Code - Module 3-Core Java - ReflectionDemo

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: Q39_ReflectionDemo.java

```
1 | import java.lang.reflect.Constructor;
2 | import java.lang.reflect.Method;
3 | import java.lang.reflect.Parameter;
4 |
5 | class ReflectionTarget {
6 |     public void greet(String name) {
7 |         System.out.println("ReflectionTarget: Hello, " + name + "!");
8 |     }
9 |
10 |    public int add(int a, int b) {
11 |        return a + b;
12 |    }
13 |
14 |    private void secretMethod() {
15 |        System.out.println("ReflectionTarget: This is a private secret method!");
16 |    }
17 | }
18 |
19 | public class Q39_ReflectionDemo {
20 |     public static void main(String[] args) {
21 |         System.out.println("=== Java Reflection Demonstration ===");
22 |
23 |         String className = "ReflectionTarget";
24 |
25 |         try {
26 |             // 1. Load the class using Class.forName()
27 |             Class<?> clazz = Class.forName(className);
28 |             System.out.println("Successfully loaded class: " + clazz.getName());
29 |
30 |             // 2. Print all declared methods and their parameters
31 |             System.out.println("\n--- Declared Methods ---");
32 |             Method[] methods = clazz.getDeclaredMethods();
33 |             for (Method method : methods) {
34 |                 System.out.print("Method Name: " + method.getName() + " | Return Type: " +
method.getReturnType().getSimpleName() + " | Parameters: ");
35 |
36 |                 Parameter[] parameters = method.getParameters();
37 |                 if (parameters.length == 0) {
38 |                     System.out.print("None");
39 |                 } else {
40 |                     for (int i = 0; i < parameters.length; i++) {
41 |                         System.out.print(parameters[i].getType().getSimpleName() + " " +
parameters[i].getName());
```

Solution Code - Module 3-Core Java - ReflectionDemo

```
42 |             if (i < parameters.length - 1) System.out.print(", ");
43 |         }
44 |     }
45 |     System.out.println();
46 | }
47 |
48 | // 3. Create an instance dynamically
49 | Constructor<?> constructor = clazz.getDeclaredConstructor();
50 | Object instance = constructor.newInstance();
51 |
52 | // 4. Invoke methods dynamically using invoke()
53 | System.out.println("\n--- Invoking Methods Dynamically ---");
54 |
55 | // Invoke public greet(String)
56 | Method greetMethod = clazz.getMethod("greet", String.class);
57 | System.out.println("Invoking greet(\"Alice\")...");
58 | greetMethod.invoke(instance, "Alice");
59 |
60 | // Invoke public add(int, int)
61 | Method addMethod = clazz.getMethod("add", int.class, int.class);
62 | System.out.println("\nInvoking add(15, 27)...");
63 | Object sumResult = addMethod.invoke(instance, 15, 27);
64 | System.out.println("Result returned from add: " + sumResult);
65 |
66 | // Invoke private secretMethod() (Requires setting accessible to true)
67 | Method secretMethod = clazz.getDeclaredMethod("secretMethod");
68 | System.out.println("\nInvoking private secretMethod()...");
69 | secretMethod.setAccessible(true); // Bypass private access modifier!
70 | secretMethod.invoke(instance);
71 |
72 | } catch (Exception e) {
73 |     System.err.println("Reflection failed: " + e.getMessage());
74 |     e.printStackTrace();
75 | }
76 | }
77 | }
```